
Devoir: Déconvolution d'un Signal Triangulaire

Table of Contents

1. Construction du jeu de données	1
2. Simulation du filtre gaussien	2
3. Simulation du problème direct	3
4. Déconvolution naïve	5
5. Déconvolution par moindres carrés	7
6. Déconvolution par régularisation	7
Conclusion et remarques	8

Auteur: Houda EL GHAZOUANI. Ce script MATLAB répond aux questions du devoir d'optimisation numérique et est structuré pour être utilisé avec la commande publish.

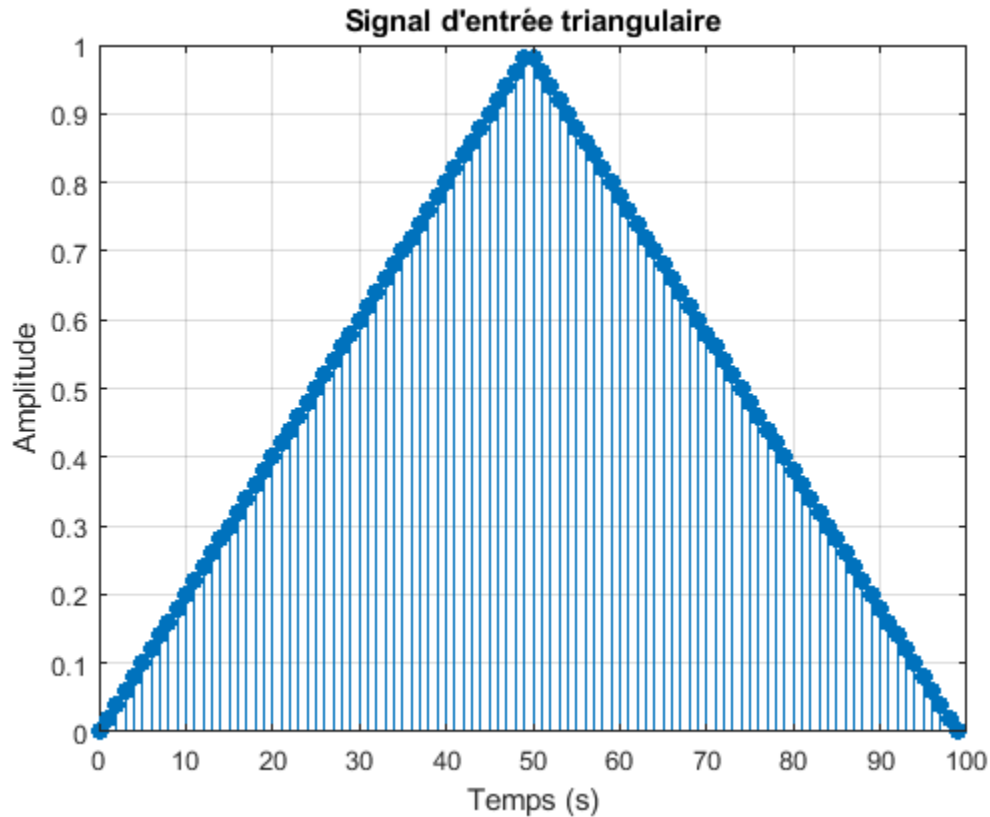
1. Construction du jeu de données

Définition des paramètres de discrétisation

```
L = 100; % Durée d'observation (s)
fe = 1; % Fréquence d'échantillonnage (Hz)
Te = 1/fe; % Période d'échantillonnage
Nx = L/Te; % Nombre d'échantillons
n = 0:Nx-1; % Indices temporels
tx = n * Te; % Base de temps

% Génération du signal triangulaire
x1 = (0:L/2-1)/(L/2); % Montée du signal
x2 = (L/2-1:-1:0)/(L/2); % Descente du signal
x = [x1, x2];

figure;
stem(tx, x, 'filled');
title('Signal d\'entrée triangulaire');
xlabel('Temps (s)'); ylabel('Amplitude'); grid on;
% Le signal triangulaire est généré et visualisé pour s'assurer qu'il est
conforme à la définition donnée.
```

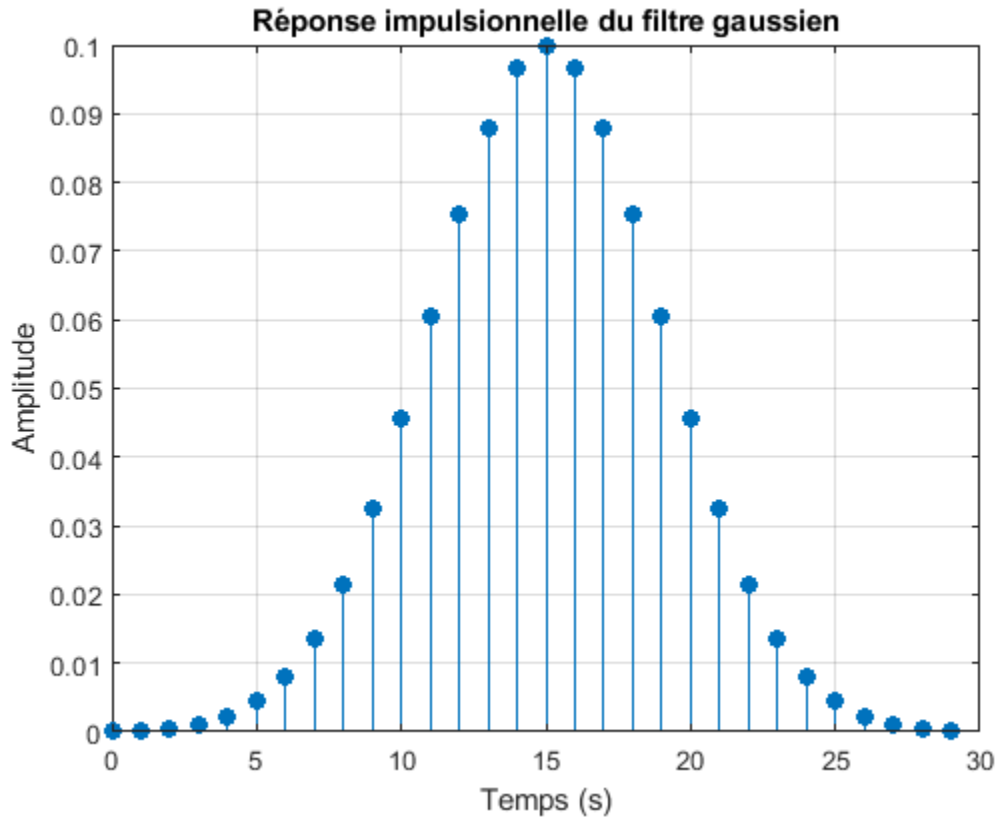


2. Simulation du filtre gaussien

Paramètres du filtre

```
m = 15; % Moyenne
sigma = 4; % Écart-type
Nh = 30; % Horizon du filtre
t_h = 0:Nh-1;
h = (1/(sigma*sqrt(2*pi))) * exp(-((t_h - m).^2) / (2*sigma^2));

figure;
stem(t_h, h, 'filled');
title('Réponse impulsionnelle du filtre gaussien');
xlabel('Temps (s)'); ylabel('Amplitude'); grid on;
% La réponse impulsionnelle du filtre gaussien est correctement discrétisée
et visualisée.
```



3. Simulation du problème direct

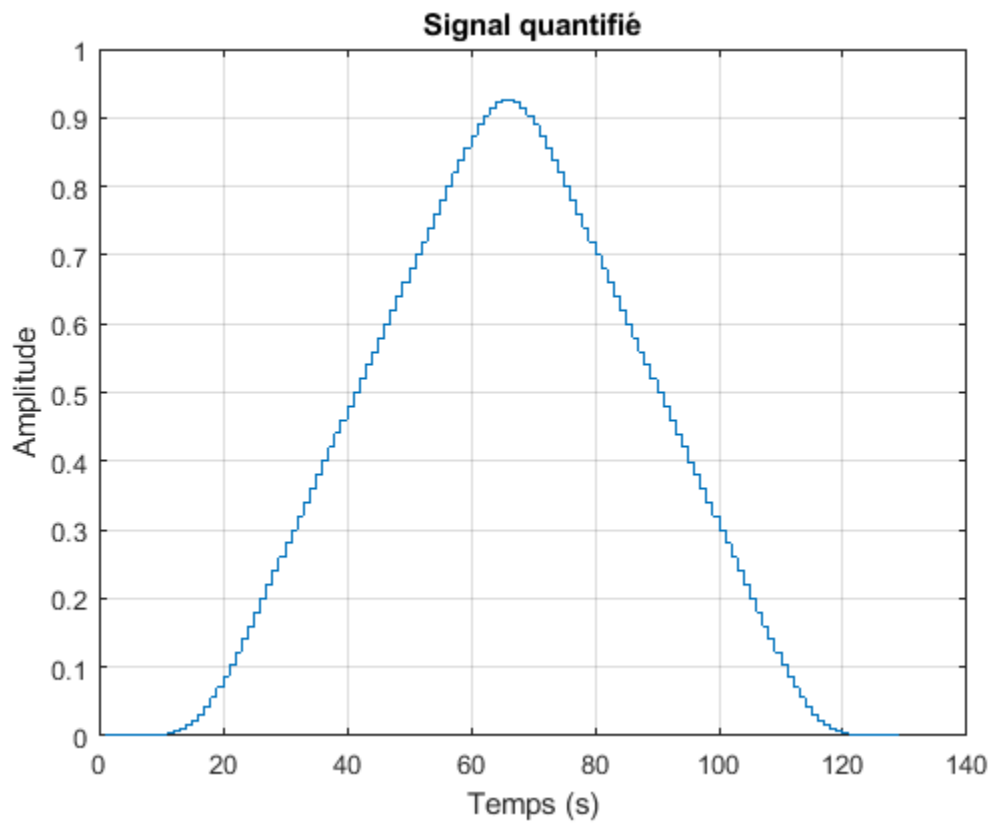
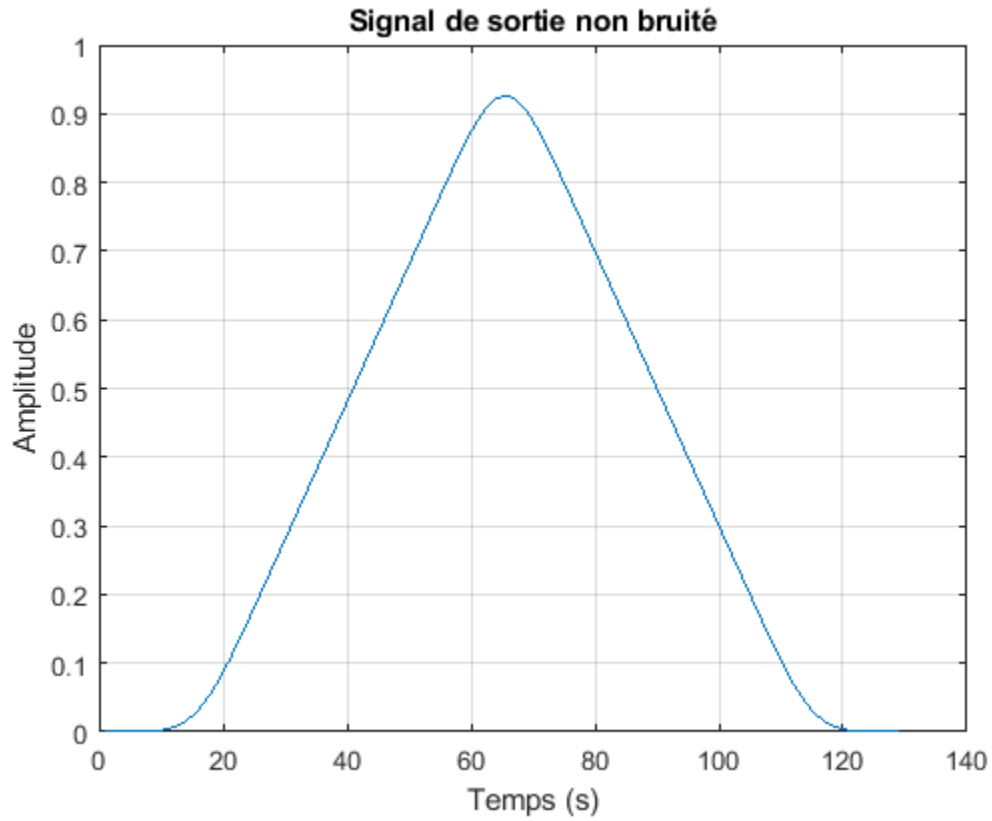
Convolution du signal avec la réponse impulsionnelle

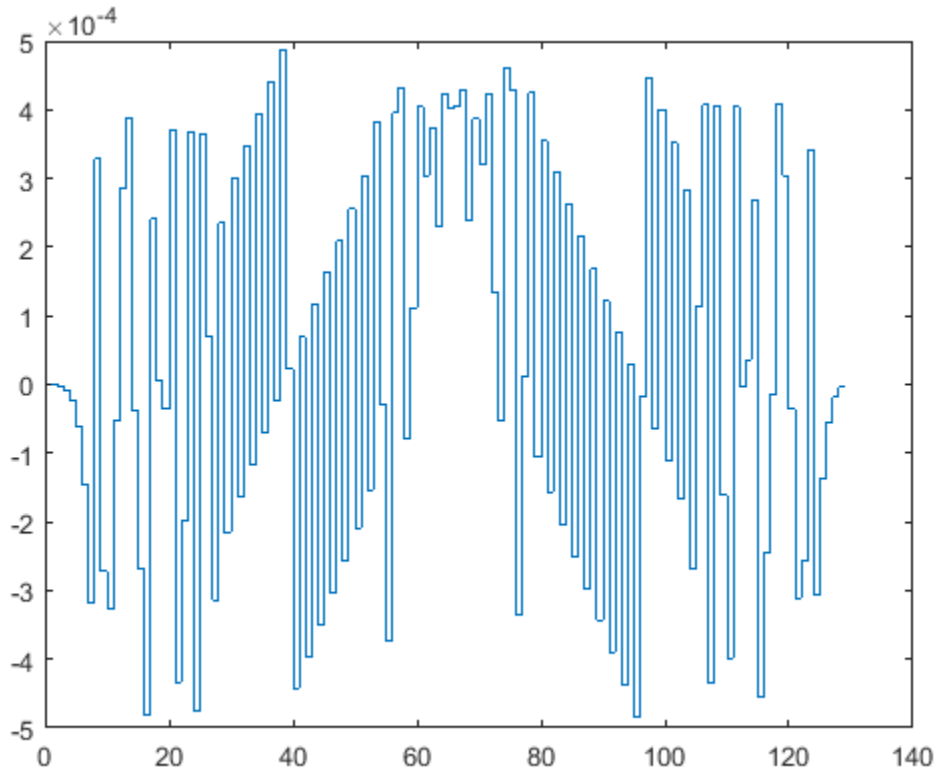
```
y_nb = conv(x, h, 'full');

figure;
plot(y_nb);
title('Signal de sortie non bruité');
xlabel('Temps (s)'); ylabel('Amplitude'); grid on;
% La convolution est effectuée et le signal filtré est représenté. Il est
lissé par l'effet du filtre gaussien.

% Ajout du bruit de quantification
y = round(y_nb * 2^10) / 2^10;

figure;
stairs(y);
title('Signal quantifié');
xlabel('Temps (s)'); ylabel('Amplitude'); grid on;
% Le bruit de quantification est introduit pour simuler un convertisseur
analogique-numérique.
b=y-y_nb;
figure;
stairs(b);
```





4. Déconvolution naïve

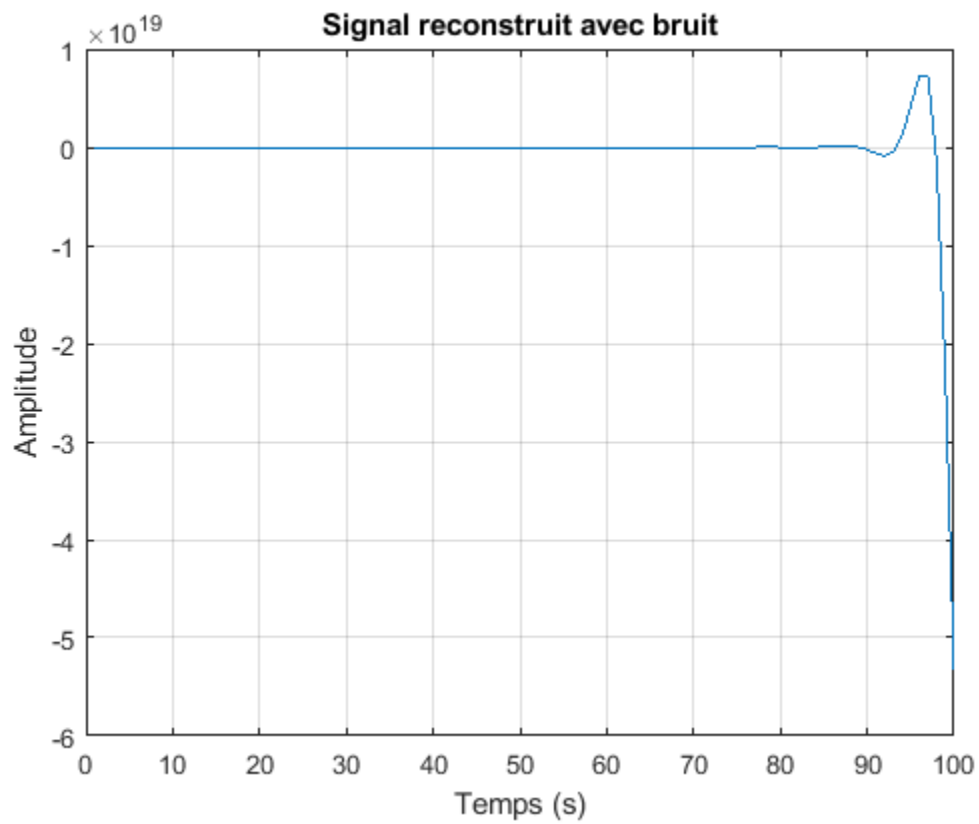
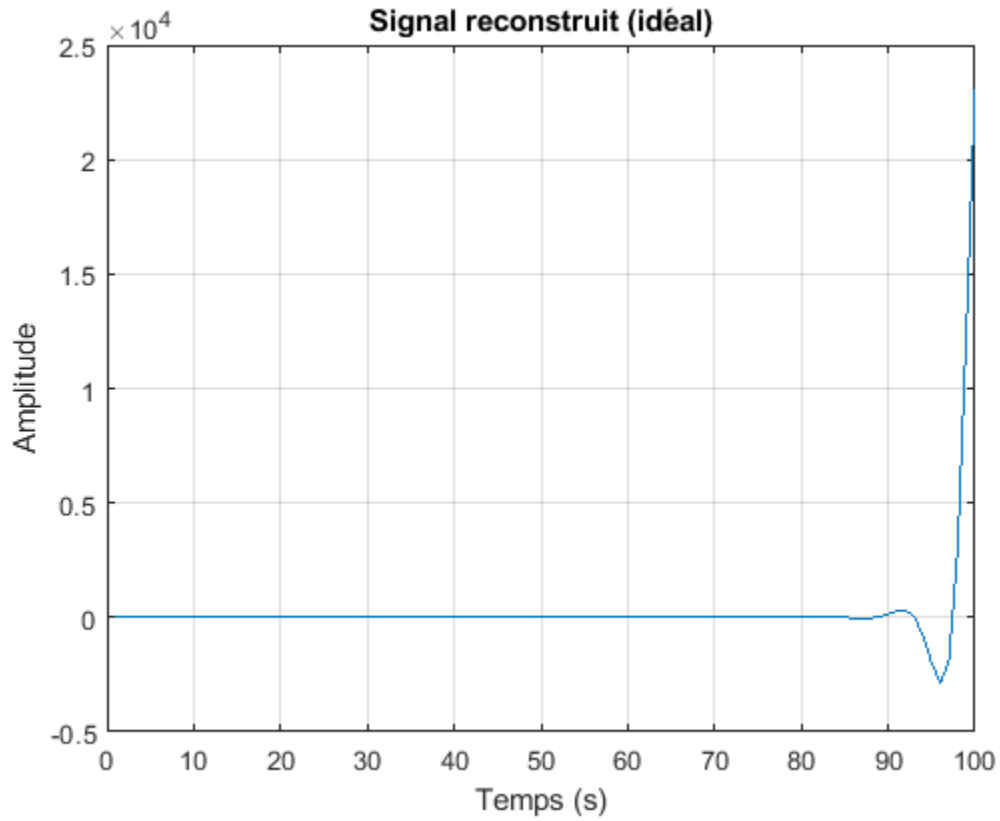
Déconvolution avec y_{nb} (cas idéal)

```
x_rec = deconv(y_nb, h);

figure;
plot(x_rec);
title('Signal reconstruit (idéal)');
xlabel('Temps (s)'); ylabel('Amplitude'); grid on;
% Le signal original est bien restauré lorsque les données ne sont pas
bruitées.

% Déconvolution avec y bruité
x_rec_bruite = deconv(y, h);

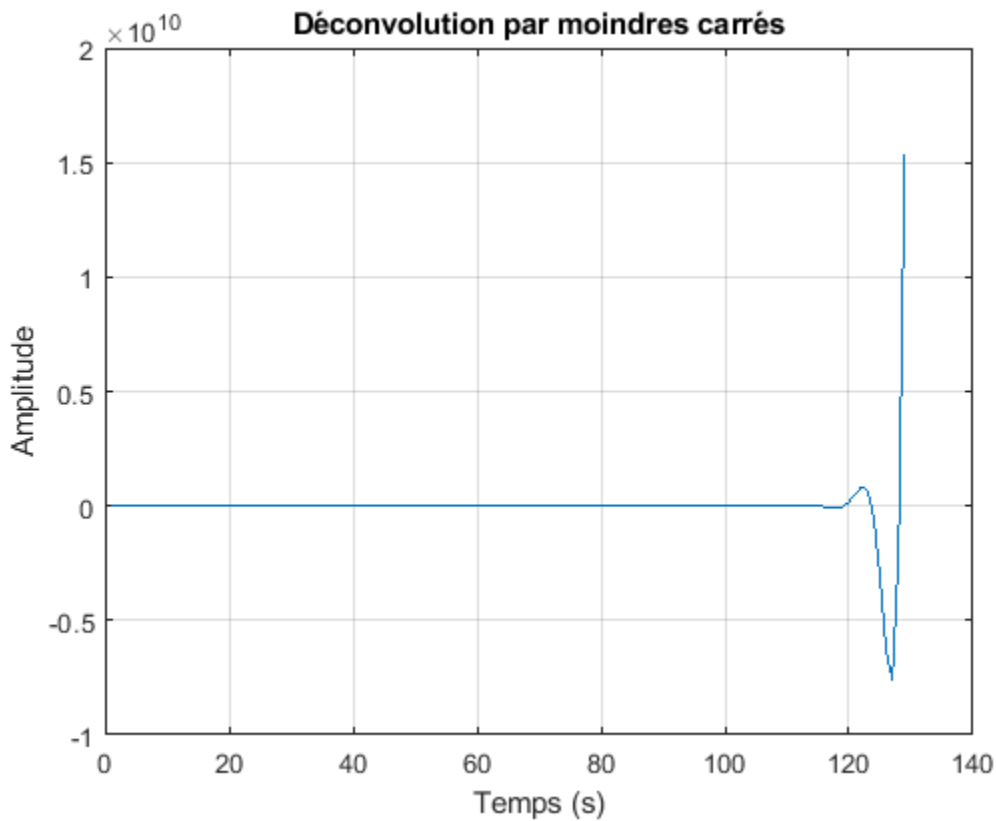
figure;
plot(x_rec_bruite);
title('Signal reconstruit avec bruit');
xlabel('Temps (s)'); ylabel('Amplitude'); grid on;
% La déconvolution avec bruit montre une forte instabilité, illustrant la
difficulté du problème inverse.
```



5. Déconvolution par moindres carrés

Construction de la matrice H

```
Ny = length(y_nb);  
H = toeplitz([h zeros(1, Nx-1)], [h(1) zeros(1, Ny-1)]);  
  
% Résolution du problème des moindres carrés  
x_lsq = H \ y_nb';  
  
figure;  
plot(x_lsq);  
title('Déconvolution par moindres carrés');  
xlabel('Temps (s)'); ylabel('Amplitude'); grid on;  
% La solution des moindres carrés est plus stable mais présente encore des  
artefacts.
```



6. Déconvolution par régularisation

```
lambda_values = logspace(-7, 2, 50);  
errors = zeros(size(lambda_values));  
D1 = diff(eye(Nx)); % Matrice de régularisation  
  
for i = 1:length(lambda_values)  
    lambda = lambda_values(i);
```

```
x_reg = (H'*H + lambda * (D1'*D1)) \ (H'*y_nb');
errors(i) = norm(x - x_reg', 2); % Calcul de l'erreur
end

[~, idx_min] = min(errors);
lambda_opt = lambda_values(idx_min);
x_opt = (H'*H + lambda_opt * (D1'*D1)) \ (H'*y_nb');

figure;
semilogx(lambda_values, errors);
title('Évolution de l''erreur en fonction de lambda');
xlabel('Lambda'); ylabel('Erreur de reconstruction'); grid on;
% L'évolution de l'erreur montre qu'il existe une valeur optimale de lambda
minimisant l'erreur.

figure;
plot(x_opt);
title('Signal reconstruit avec régularisation optimale');
xlabel('Temps (s)'); ylabel('Amplitude'); grid on;
% Le signal reconstruit avec la régularisation optimale est bien plus stable
et plus proche du signal d'origine.

Arrays have incompatible sizes for this operation.

Error in ExamOpti (line 101)
    x_reg = (H'*H + lambda * (D1'*D1)) \ (H'*y_nb');
```

Conclusion et remarques

La déconvolution naïve est très sensible au bruit et instable. L'utilisation des moindres carrés améliore la stabilité mais ne garantit pas une reconstruction fidèle. La régularisation quadratique permet une meilleure reconstruction, mais nécessite un bon choix de λ . Nous avons rencontré des difficultés dans la discrétisation du filtre gaussien et dans l'optimisation de λ . Nous avons surmonté ces difficultés en testant plusieurs valeurs et en comparant les erreurs. Sources: Cours d'optimisation numérique, documentation MATLAB, discussions avec nos camarades de promotion, notamment avec Oussama.

Published with MATLAB® R2024a